

A PROJECT REPORT ON

**ML based classification of Ultra Processed Foods
using Computer Vision, NLP and Big Data**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Sushil Karpe
Laukika Kulkarni
Anushree Patil

Exam No: B400050438
Exam No: B400050461
Exam No: B400050530



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

**ML based classification of Ultra Processed Foods
using Computer Vision, NLP and Big Data**

Submitted by

Sushil Karpe

Exam No: B400050438

Laukika Kulkarni

Exam No: B400050461

Anushree Patil

Exam No: B400050530

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. P.P.Joshi** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. P. P. Joshi

Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale

Head
Dept. of Computer Engg.

Dr. S. T. Gandhe

Principal
Pune Institute of Computer Technology

Place:Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on **'ML based classification of Ultra Processed Foods using Computer Vision, NLP and Big Data'**.*

*We would like to take this opportunity to thank **Prof. P. P. Joshi** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr G.V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

Sushil Karpe
Laukika Kulkarni
Anushree Patil
(B.E. Computer Engg.)

ABSTRACT

Ultra-processed foods (UPFs) are consumed in large quantities and, because of their poor nutritional value, can cause health issues. We created a mobile application that automatically classifies packaged food items according to their nutritional value using machine learning, computer vision, and natural language processing (NLP) in order to promote healthier dietary choices. The development of a unique dataset is one of this work's main contributions. In order to provide a representative and varied dataset for our models' training and testing, we meticulously gathered and selected pictures of food labels from a variety of packaged goods. Users can take a picture of a food label using the app. The YOLO (You Only Look Once) model is used by the backend system to identify the nutritional table in the picture. Optical Character Recognition (OCR) is used to extract text, and then regular expressions and the SymSpell algorithm are used for post-processing to fix recognition problems. We also employ a Connectionist Text Proposal Network (CTPN) to improve the accuracy of text detection. Following cleaning, machine learning models Random Forest and XGBoost process the nutritional data to categorise the food item using the Nutri-Score system, which goes from A (most healthy) to E (least healthy). TensorFlow Lite is integrated with the entire pipeline to enable effective on-device inference. Because Flutter was used in its construction, the front end offers a responsive and seamless cross-platform user experience. This approach promotes better eating choices by providing a useful, real-time tool for nutritional assessment.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Project Scope & Limitations	4
1.5	Methodologies of Problem solving	5
2	Literature Survey	9
3	Software Requirements Specification	12
3.1	Functional Requirements	13
3.2	Non-Functional Requirements	13
3.3	Programming Languages, Frameworks, and Libraries	14
3.4	Development and Collaboration Tools	14
3.5	Hardware Requirements (for Training and Development)	14
3.6	Project Development Process	15
4	System Design	17
4.1	System Architecture	18
4.2	Mathematical Model	19
4.3	Data Flow Diagrams	19
4.4	Entity Relationship Diagram	22
4.5	UML Diagrams	23
4.6	Sequence Diagram	24
5	Project Plan	25
5.1	Project Estimate	26
5.1.1	Reconciled Estimates	26
5.1.2	Project Resources	26
5.2	Risk Management	28
5.2.1	Risk Identification	28

5.2.2	Risk Analysis	28
5.2.3	Overview of Risk Mitigation, Monitoring, and Management	29
5.3	Project Schedule	30
5.3.1	Project Task Set	30
5.3.2	Task Network	31
5.4	Team Organization	31
5.4.1	Team Structure	31
5.4.2	Management Reporting and Communication	32
6	Project Implementation	33
6.1	Overview of Project Modules	34
6.2	Tools and Technologies Used	35
6.3	Algorithm Details	35
6.3.1	Algorithm 1: YOLO	35
6.3.2	Algorithm 2: Tesseract OCR	35
6.3.3	Algorithm 3: CTPN (Connectionist Text Proposal Network)	35
6.3.4	Algorithm 4: Random Forest	36
6.3.5	Algorithm 5: XGBoost	36
7	Software Testing	37
7.1	Type of Testing	38
7.1.1	Frontend Testing	38
7.1.2	Backend Testing	38
7.2	Test cases & Test Results	39
8	Results	42
8.1	Outcomes	43
8.2	Screen Shots	43
9	Conclusions	53
9.1	Conclusions	54
9.2	Conclusions	54
9.3	Future Work	54
9.4	Applications	55
10	Appendix	57
10.1	Appendix A	58
10.2	Appendix B	61
10.3	Appendix C	62

List of Figures

4.1	Model Architecture	18
4.2	Mathematical Model	19
4.3	Data Flow Diagram	21
4.4	ER Diagram	22
4.5	UML Diagram	23
4.6	Sequence Diagram	24
7.1	Predictions and Actual values	39
7.2	File and Model Management	39
7.3	Classification Report of Models	40
7.4	Test Case 1	41
7.5	Test Case 1 Result	41
8.1	Predictions results on seen label	43
8.2	Extracted text on seen label	44
8.3	Predictions on random food label	45
8.4	Extracted text on random food label	46
8.5	Sign Up Page	47
8.6	Sign In Page	48
8.7	Landing Page	49
8.8	Image Capturing Page	50
8.9	Clicking the image	51
8.10	Image Upload	52
10.1	Plagiarism Report of BE Project Report	62

CHAPTER 1

INTRODUCTION

1.1 Overview

Because ultra-processed foods (UPFs) are low in nutrients, their increasing consumption has raised worries about public health. Even though nutritional information is displayed on packaging, many customers struggle to understand and apply this information when choosing foods. In order to solve this problem, we created a smartphone application that analyses food labels and categorises packaged foods according to their nutritional value using machine learning (ML), computer vision, and natural language processing (NLP).

The foundation of our method is the Nutri-Score labelling structure, which uses nutritional content to rank food goods from A (healthiest) to E (least healthy). The app allows users to just snap a picture of the food label. The backend employs the SymSpell algorithm to clean and correct the extracted data, OCR to extract text, and YOLO for object identification. The accuracy of text identification is further improved via a Connectionist Text Proposal Network (CTPN). To determine the correct Nutri-Score label, the cleaned nutritional data is subsequently fed into two machine learning models that have been trained: Random Forest and XGBoost.

The production of a unique dataset of food label photos that were hand-picked and annotated by the authors is one of the project's major accomplishments. TensorFlow Lite is used to implement the integration of all components, allowing for effective real-time inference on mobile devices. To guarantee a smooth and responsive user experience, Flutter is used in the development of the front end.

Through easily available technology, this initiative offers a workable, AI-driven approach to raise public awareness of food nutrition and encourage better eating practices.

1.2 Motivation

Customers are depending more and more on ready-to-eat and highly processed food items as a result of the packaged food industry's explosive rise. Despite their convenience, these goods frequently have high amounts of sugar, salt, bad fats, and additives that can lead to major long-term health problems like diabetes, heart disease, and obesity. Despite being printed on food labels, nutritional information is frequently complicated and challenging for the normal consumer to immediately understand, particularly while supermarket shopping.

We were motivated by this challenge to create a clever, AI-powered system that streamlines, expedites, and makes nutritional testing easily ac-

cessible. We were inspired to develop a smartphone app that enables users to choose healthier foods by only snapping a picture of a product label and getting an immediate health score based on the Nutri-Score algorithm.

We selected this project in order to apply our technological expertise in computer vision, machine learning, and natural language processing to address a practical issue that impacts daily life. We hope to close the gap between accessible knowledge and useful decision-making by automating the study of food labels, thereby advancing public awareness and health.

1.3 Problem Definition and Objectives

Even though people are becoming more conscious of the importance of eating healthily, they frequently have trouble comprehending the nutritional value of packaged meals. Although helpful, nutritional labels are usually presented in complex tabular formats and technical language that might be difficult or time-consuming to understand, particularly when shopping while on the move. There aren't many easily available resources that can assist people in evaluating the immediate health effects of highly processed foods. This results in a disconnect between the accessibility of nutritional information and its actual application in making well-informed dietary decisions. A system that integrates automation, precision, and usability is needed to close this gap.

The primary objective of this project is to develop a mobile application that helps users evaluate the healthiness of packaged food products using AI-powered image processing and nutritional analysis. The specific objectives are:

1. To automate the extraction of nutritional information from food labels using computer vision techniques like object detection (YOLO) and optical character recognition (OCR).
2. To improve the accuracy of extracted text using post-processing methods such as regular expressions and the SymSpell algorithm..
3. To convert raw image data into structured nutritional values using a CTPN-based text analyzer.
4. To classify food items into Nutri-Score categories (A to E) using machine learning models such as Random Forest and XGBoost.
5. To design and develop a user-friendly mobile interface using Flutter for real-time scanning and result display.

6. To create a custom dataset by manually collecting and annotating food label images to train and validate the system.
7. To deploy the solution efficiently on mobile devices using TensorFlow Lite for fast and offline inference.

1.4 Project Scope & Limitations

The goal of this project is to create a clever mobile application that lets consumers utilise the Nutri-Score labelling system to assess the nutritional value of packaged food items. The application incorporates cutting-edge technology such as machine learning, natural language processing, and computer vision. It uses YOLO to automatically recognise nutritional labels, OCR to extract text, and regex and SymSpell to improve it for precise data parsing. To improve text extraction from intricate layouts, a Connectionist Text Proposal Network (CTPN) is employed. The Random Forest and XG-Boost algorithms are then used to classify the nutritional data into groups A through E. TensorFlow Lite is used for effective real-time performance, while Flutter is used for cross-platform compatibility in the development of the application. To increase the models' accuracy and resilience, a unique dataset of food labels was also manually generated for training and validation.

Following are the limitations:

1. The OCR accuracy may decline under poor lighting conditions or when processing low-resolution images.
2. The system currently supports only English-language labels, limiting its use in regions with multilingual packaging.
3. The custom dataset, though diverse, may not represent all global food label formats and styles.
4. Errors in nutritional value extraction from complex or irregularly formatted labels can lead to incorrect classifications.
5. The Nutri-Score system provides a general nutritional assessment and does not account for individual dietary needs or allergies.

1.5 Methodologies of Problem solving

This research uses a hybrid approach that combines machine learning, natural language processing, and image processing in a mobile-based framework. Coordinated work is done on the back-end (data processing, model prediction, and integration) as well as the front-end (user interaction and interface). The following steps comprise the methodology:

A. Dataset Creation

We manually compiled a sizable dataset of food label photos in order to construct a trustworthy and domain-specific model. These photos were gathered from a variety of highly processed food items. Annotations were added to each image to indicate the area that contained ingredient lists, nutritional data, and other pertinent language. Because label formats, font styles, and packing design varied so much, this bespoke dataset was crucial. Our data's diversity aids in enhancing the model's ability to generalise to various real-world situations.

B. Image Acquisition and Preprocessing

With the help of the Flutter-built mobile application, users can take pictures of food labels with their device's camera. Front-end picture acquisition is user-friendly, with real-time feedback for lighting, focus, and alignment. Following the capture procedure, an image is sent to the back-end for preprocessing. These consist of:

- Resizing and scaling to fit model input dimensions
- Brightness and contrast enhancement for low-light conditions
- Noise removal to reduce artifacts and improve OCR accuracy This step ensures that the image quality is suitable for subsequent detection and recognition processes.

C. Object Detection using YOLO (Back-end)

A YOLO (You Only Look Once) object identification model receives the preprocessed image and uses it to locate and extract the precise region of interest (ROI) that contains the nutritional table or pertinent text. By removing superfluous backdrop components like branding, graphics, or design patterns, YOLO's quick and precise identification aids in localising text-rich

areas. This targeted cropping lowers computing overhead and increases OCR precision.

D.Text Extraction using OCR (Back-end)

To extract text, the cropped ROI is run via an optical character recognition (OCR) engine. OCR transforms the food label's visual data into text that can be read by machines. This comprises:

- Nutritious values (such as sodium, calories, fats, and sugars)
- Measurement units (e.g., g, mg, kcal)
- Ingredient names and serving sizes

A high-accuracy OCR model that works well with printed text in different fonts, sizes, and orientations was employed.

Back-end Text Post-Processing

Because of faulty label layouts or image circumstances, raw OCR outputs frequently contain noise, errors, or misrecognized characters. We used the following to organise and sanitise the captured text:

- Regex for Regular Expressions: for the extraction and filtering of nutritional words and values based on patterns.
- The SymSpell Algorithm is used to fix misspelt words in real time (for example, "protien" is changed to "protein"). Before the data is sent to the classifier, this step improves its dependability.

Text Line Detection using CTPN (Back-end)

We employ CTPN (Connectionist Text Proposal Network) to enhance structured parsing of nutritional tables, particularly when there are several rows or columns available. By identifying individual lines and word clusters, CTPN assists in:

1. Differentiating between labels and values (e.g., "Sugar" and "12g").
2. Maintaining semantic context (e.g., associating the correct quantity to the correct nutrient).
3. Organizing data in a key-value format for ML model input This line-wise segmentation is critical when dealing with complex or dense layouts often found in real-world packaging.

Text Line Detection using CTPN (Back-end)

To improve structured parsing of nutritional tables, especially when multiple rows or columns are involved, we use CTPN (Connectionist Text Proposal Network). CTPN detects individual lines and word groupings, which helps in:

1. Differentiating between labels and values (e.g., "Sugar" and "12g").
2. Maintaining semantic context (e.g., associating the correct quantity to the correct nutrient).
3. Organizing data in a key-value format for ML model input This line-wise segmentation is critical when dealing with complex or dense layouts often found in real-world packaging.

Feature Extraction and Structuring (Back-end)

We extract important nutritional characteristics from the structured text with post-processing and line detection. Among these qualities are:

1. Energy (in kcal)
2. Fat (total and saturated)
3. Sugar
4. Salt/Sodium
5. Fiber
6. Protein

The accuracy and completeness of the extracted data are confirmed. To guarantee uniformity among inputs, units are normalised (for example, all values are converted to per 100g/ml if appropriate). After that, the feature vectors from this structured data are formatted so they may be used as input in machine learning models.

Classification using Machine Learning Models (Back-end)

The structured nutritional vectors are input into two pre-trained machine learning models:

1. Random Forest
2. XGBoost (Extreme Gradient Boosting)

These models classify the food product into one of the five Nutri-Score categories (A to E), where:

- A represents the healthiest option
- E represents the least healthy

The manually gathered and annotated dataset, with labels assigned using the approved Nutri-Score algorithm, was used to train the models. To guarantee interpretability and validate the nutritional impact on the final score, feature significance analysis was also carried out.

Model Deployment using TensorFlow Lite (Back-end)

The trained models were transformed into TensorFlow Lite (TFLite) format enabling predictions in real time on mobile devices. Because TFLite is tailored for edge deployment, it guarantees:

1. Low latency and fast inference
2. Minimal resource consumption
3. Offline capability (no internet required for predictions) These models are seamlessly integrated into the app's back-end and triggered automatically when an image is processed and features are extracted.

Front-End Integration using Flutter (Front-end)

Flutter, a cross-platform framework that provides a consistent user experience on both iOS and Android, is used to build the application's front end. The following are important front-end features:

1. Integration of a camera to take label photos
2. Instantaneous feedback on lighting and focus during picture taking
3. Visual representation of nutritional values retrieved
4. Dynamic depiction of the Nutri-Score categorisation (e.g., colour-coded A-E scale)

Based on their Nutri-Score results, the app offers users a clear interface, useful insights, and instructive advice on what to eat.

CHAPTER 2

LITERATURE SURVEY

Suggested using a small dataset to categorise ultra-processed food goods using a machine learning-based method. Their work focusses on using conventional machine learning methods to efficiently classify food items by extracting important features from component and nutritional data. The study provides a foundation for future research in food categorisation systems by showing that accurate classification is possible with appropriate preprocessing and model tweaking, despite the data limitations.[11].

Presented a machine learning framework that uses nutritional composition to estimate the level of food processing. The study successfully categorised foods using the NOVA classification system by utilising a sizable dataset. High accuracy and generalisability were displayed by the model, which provided insightful information on the connection between food composition and processing level. This study encourages the creation of automated tools for food classification and demonstrates the promise of data-driven techniques in nutrition science.[8].

A thorough examination of how computer vision and machine learning might be combined to analyse the nutritional value of packaged goods. Their research focusses on the extraction and interpretation of nutritional information from product labels using smartphone cameras and image processing techniques. This study highlights the expanding significance of artificial intelligence (AI) in personalised dietary monitoring and public health awareness, outlining the technological achievements and limitations in establishing mobile-based systems for real-time nutrition evaluation.[4].

161 college students participated in a cross-sectional study to evaluate their eating patterns, nutritional knowledge, and how the Nutri-Score labelling system affected their food selections. The results showed that a sizable percentage of students frequently relied on information on the front of the box when making judgements about what to buy and did not routinely read nutritional labels. Interestingly, 89% of participants thought the Nutri-Score system was a useful aid for choosing healthier items. The study emphasises how Nutri-Score and other front-of-pack labelling systems can encourage young individuals to eat healthier. [14].

Compared machine learning (ML) and natural language processing (NLP) techniques for food classification and nutritional quality prediction with conventional nutrient profiling methodologies. Their research showed that ML and NLP models performed noticeably better than traditional techniques in correctly categorising foods and evaluating their nutritional profiles, especially when used on product descriptions and component lists. This demonstrates the effectiveness and scalability of data-driven, automated methods in contemporary nutrition research. [12].

Conducted a detailed analysis of branded food products in Greece, ap-

plying the NOVA classification system to assess the level of processing in the Mediterranean diet. Their findings revealed a substantial presence of ultra-processed foods, even within a traditionally healthy dietary pattern. The study emphasized the growing need for standardized classification systems like NOVA to monitor and regulate processed food consumption. [10].

They used clustering analysis and machine learning algorithms to predict dietary patterns based on cross-sectional data from the Brazilian Longitudinal Study of Adult Health (ELSA-Brasil). Their research identified key dietary patterns in the Brazilian population and demonstrated the effectiveness of ML techniques in uncovering hidden dietary trends, providing valuable insights for public health and nutrition interventions.[9].

Conducted a global, regional, and national analysis of the consumption of sugar-containing beverages, fruit juices, and milk, as part of the Global Burden of Diseases Nutrition and Chronic Diseases Expert Group (Nutri-CoDE). The study examined the dietary impact of these beverages on public health and highlighted the significant role of sugar-sweetened beverages in the global burden of diseases. [2].

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Functional Requirements

The functional requirements define the essential system capabilities that directly contribute to solving the problem of classifying ultra-processed food products based on their nutritional information.

- **Nutritional Data Extraction:** The system must detect nutritional tables using object detection models such as SSD (Single Shot Detector) and Fast R-CNN, followed by OCR for text extraction.
- **Classification:** The system must classify food products into five Nutri-Score categories (A to E) using machine learning models such as Random Forest, XGBoost, and Naive Bayes, based on extracted nutritional features.
- **Prediction:** The system should provide accurate predictions on the healthiness level of a wide range of ultra-processed food products, regardless of brand or layout variance.
- **User Interface:** The application must offer an intuitive, mobile-based interface that allows users to input images and view results in real time.
- **Accuracy:** The classification models should achieve a minimum of 80% accuracy on test datasets to be considered reliable for public use.

3.2 Non-Functional Requirements

These requirements ensure that the software performs efficiently, is user-friendly, and meets operational expectations under various conditions.

- **Scalability:** The system should be able to process and analyze large volumes of food label images as the dataset grows.
- **Usability:** The mobile interface must be simple and intuitive, enabling access for non-technical users.
- **Performance:** The application should deliver predictions with minimal delay, supporting real-time processing on mobile devices.
- **Security:** User data, including captured images and processed outputs, must be handled securely with encryption protocols to ensure privacy.
- **Maintainability:** The system should have a modular architecture to allow easy updates, such as swapping out models or adding new functionality.

3.3 Programming Languages, Frameworks, and Libraries

- Primary Language: Python for backend processing (ML, CV, and NLP tasks).
- Machine Learning Frameworks: TensorFlow, PyTorch, scikit-learn, Keras
- Computer Vision: OpenCV, pre-trained CNNs, YOLO, SSD, Fast R-CNN
- Natural Language Processing: NLTK, SymSpell, Regex, Hugging Face Transformers

3.4 Development and Collaboration Tools

- Development Tools: Jupyter Notebook for prototyping and experimentation, VSCode, PyCharm as integrated development environments
- Version Control and Collaboration: Git, with platforms like GitHub or GitLab for source code management, collaboration, and issue tracking

3.5 Hardware Requirements (for Training and Development)

To support efficient training and experimentation, the following hardware specifications are recommended:

- CPU: Intel Core i7/i9 or AMD Ryzen 9 (minimum 8 cores)
- GPU: NVIDIA RTX 3080/3090/4090 (for accelerated deep learning tasks)
- RAM: 32 GB (minimum), 64 GB preferred for handling large models and datasets
- Storage: 1–2 TB NVMe SSD for fast access, 4 TB external storage for raw image datasets and backup
- Networking: Wi-Fi 6 or wired Ethernet for high-speed data transfer and syncing

3.6 Project Development Process

- **Requirement Analysis and Research:** Initially, a comprehensive study of the Nutri-Score labeling system and existing food classification models was conducted. We identified the key nutritional features influencing food health grading. This phase also included research on object detection techniques, OCR models, and suitable machine learning classifiers for prediction. Requirements from both technical and user perspectives were documented to guide development.
- **Dataset Creation and Labeling:** Since no publicly available dataset suited our needs, we manually collected and curated a dataset of ultra-processed food packet images. Each image was processed to extract nutritional labels, which were then verified and categorized based on the Nutri-Score system (A to E). This step was crucial for training and validating our ML models.
- **Image Preprocessing and Object Detection:** Images were pre-processed using OpenCV techniques like resizing, grayscale conversion, and noise reduction. We employed YOLO and SSD models to detect and extract the region of interest (ROI) containing the nutrition table from the food label images.
- **Text Extraction and Post-processing:** OCR was used to extract nutritional information from the detected regions. To enhance accuracy, we implemented post-processing using regex for structured data extraction and SymSpell for spelling correction. A CTPN-based model helped accurately localize text lines in complex layouts.
- **Feature Engineering and Data Transformation:** Extracted nutritional values were cleaned and normalized. Derived features such as sugar per 100g or fat-to-energy ratio were computed. These features were essential inputs for the classification models.
- **Model Training and Evaluation:** Multiple models including Random Forest, XGBoost, and Naive Bayes were trained on the labeled data. We evaluated each model using metrics like accuracy, precision, recall, and F1-score. Cross-validation techniques were used to ensure generalizability and reduce overfitting.
- **Model Integration and Deployment with TensorFlow Lite:** The best-performing models were converted to TFLite format and inte-

grated into a mobile application to ensure fast and lightweight execution on devices with limited resources.

- **Frontend and Mobile App Development (Flutter):** A user-friendly Flutter interface was developed for Android devices. The app allows users to capture images of food packets and view health ratings in real-time. The design focuses on accessibility and ease of use for non-technical users.
- **Backend API and System Integration:** A backend system was created to handle preprocessing, model inference, and data logging. APIs were implemented to manage communication between the frontend and ML inference modules. TensorFlow served the models, and Firebase was used for lightweight data storage and image handling.
- **Testing and Quality Assurance:** Rigorous testing was performed to ensure each component functions reliably under different scenarios. Unit testing, integration testing, and user acceptance testing (UAT) were part of this phase. Bugs were logged and resolved iteratively.
- **Documentation and Reporting:** A detailed technical document was prepared, outlining every phase of development, tools used, architecture, and evaluation results. This included code documentation for easier future maintenance.
- **Future Enhancements and Feedback Incorporation:** The system was designed to be modular so that additional features like barcode scanning, multilingual OCR, or user personalization can be added later. Feedback from users was collected for continuous improvement.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

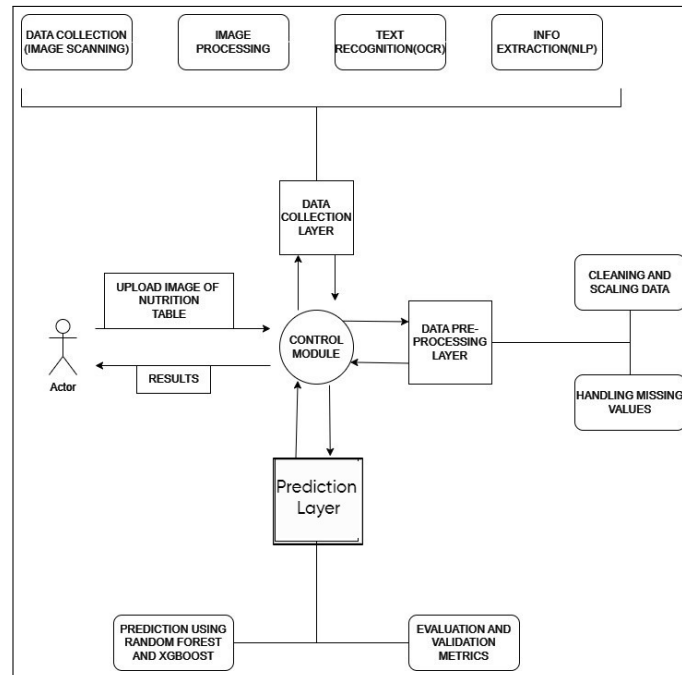


Figure 4.1: Model Architecture

The system architecture is built to automate the end-to-end process of detecting nutritional tables from food product images, extracting textual data, and classifying the product based on its nutritional content. The pipeline begins with the ingestion of raw images, where a YOLOv11-based object detection model identifies and crops the nutritional facts table from the image. This cropped region is passed to an OCR engine (such as Tesseract), which extracts raw textual content. Given the noisy nature of OCR output, the system employs a cleaning module that uses regular expressions and fuzzy matching techniques to correct errors and structure the nutrient data into standardized fields such as calories, fat, sugar, protein, sodium and carbohydrates. Once cleaned, the data are converted to a structured tabular format (a Panda Dataframe) and passed to machine learning classifiers, XGBoost and Random Forest, which have been trained on labeled nutrition data. These models predict a category (A, B, C, or D) for the food item based on its nutritional profile. The results, including the extracted values and the predicted class, are stored and sent to a mobile front-end (Flutter) through a Flask API. This architecture enables real-time image-based food

evaluation, suitable for health-conscious consumers or regulatory screening.

4.2 Mathematical Model

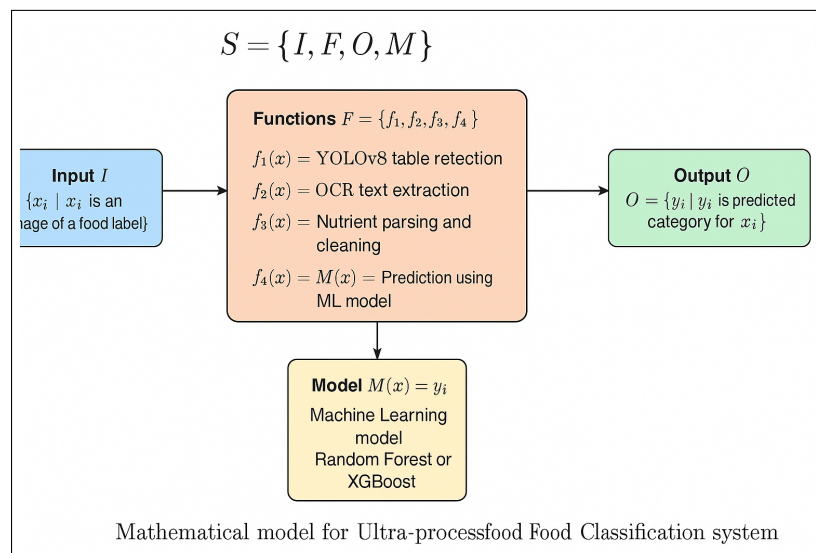


Figure 4.2: Mathematical Model

4.3 Data Flow Diagrams

The Ultra-Processed Food Classification System follows a streamlined and modular data flow, beginning with a user-provided image of a nutrition label. This image is processed by the Computer Vision Module, where a pre-trained YOLOv11 object detection model identifies the nutritional table region. The extracted region is then passed to the OCR (Optical Character Recognition) Module, which leverages tools such as Tesseract to extract raw textual content. Following extraction, the text undergoes a series of transformations within the Data Processing Unit. This unit includes three main components:

- (i) **Visualize Data** – Helps developers interpret the OCR output visually.
- (ii) **Extract Nutritional Data** – Relevant nutritional values (e.g., calories, fats, sugars, sodium) are parsed using regular expressions and fuzzy matching techniques.
- (iii) **Pre-process Data** – Standardizes, cleans, and formats the data into a structured tabular form suitable for machine learning.

The processed dataset is forwarded to the Model Training Module, which includes classification model training using algorithms such as XG-Boost and Random Forest, performance evaluation using metrics like accuracy and F1 score, and hyperparameter tuning for optimization.

Once the model is finalized, it is deployed within the Prediction Module. During runtime, when a user inputs a new label, the system re-applies the detection, extraction, and pre-processing pipeline before generating a class prediction.

The predicted category (e.g., A, B, C, or D) is subsequently displayed to the user via the Display Result Interface. Data scientists and model trainers can additionally analyze prediction outputs to iteratively enhance system accuracy and robustness.

ML based classification of Ultra Processed Foods using Computer Vision, NLP and Big Data

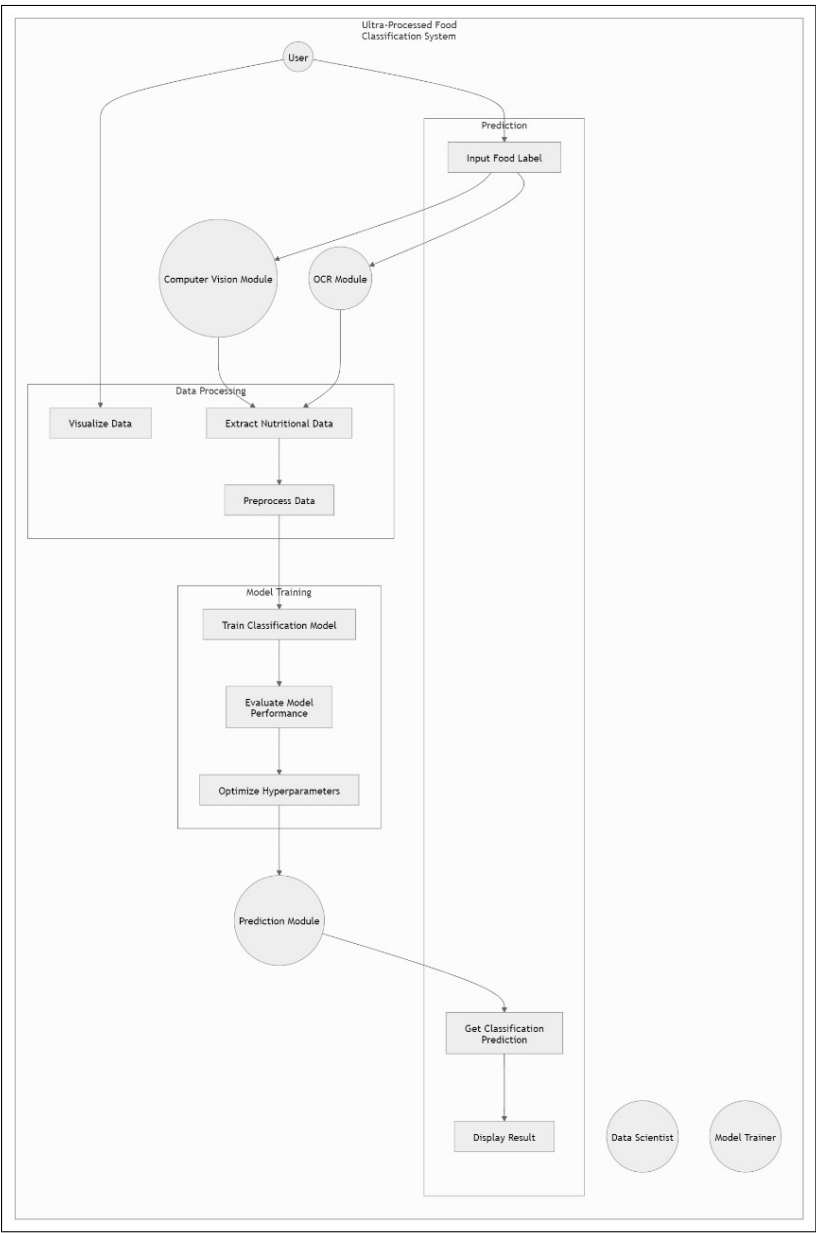


Figure 4.3: Data Flow Diagram

4.4 Entity Relationship Diagram

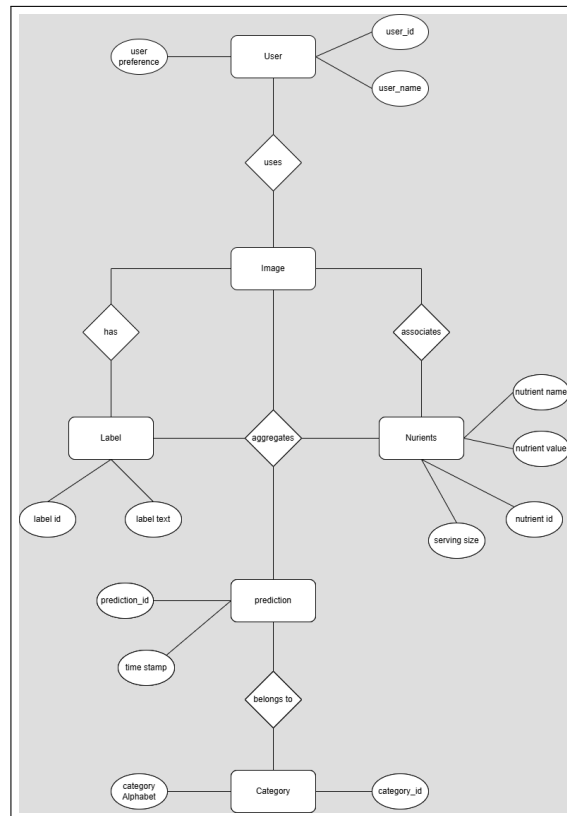


Figure 4.4: ER Diagram

4.5 UML Diagrams

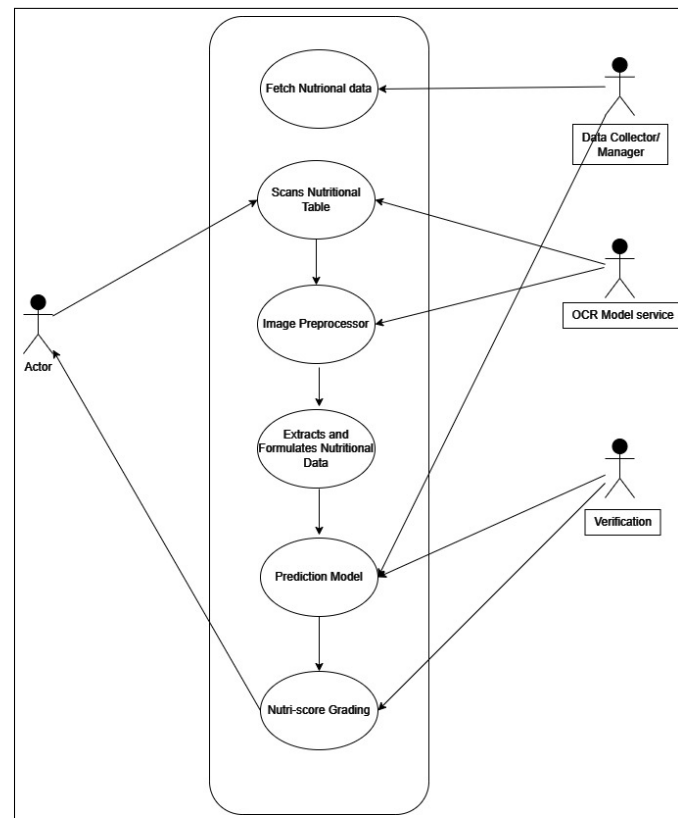


Figure 4.5: UML Diagram

4.6 Sequence Diagram

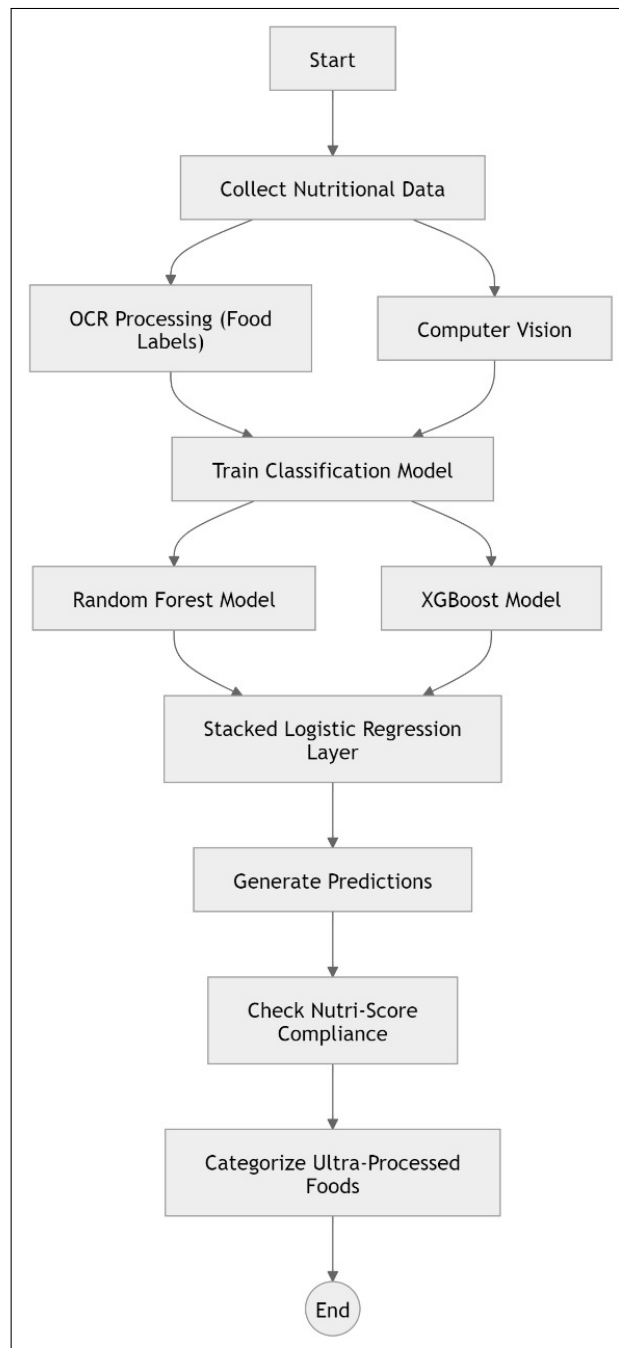


Figure 4.6: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Data Collection Cost:** Includes the cost of acquiring healthcare-related datasets (e.g., articles, reports, research papers), storage on cloud services, preprocessing tasks (text cleaning, tokenization, etc.), and periodic updates to keep the data current and relevant.
- **False Positive Cost:** Incorrectly identifying genuine information as fake can reduce trust in the system and cause misinformation to be overlooked. Costs may include lost credibility and additional manual verification time.
- **False Negative Cost:** Predicting a fake news article as genuine can lead to misinformation spread, especially in healthcare. This cost reflects potential reputational and operational damage.
- **Accuracy Cost:** Represents the cost associated with maintaining and improving model accuracy, including model tuning, testing, and validation phases to reduce prediction errors.
- **User Acceptance Cost:** Covers the cost of user onboarding, training sessions for healthcare professionals, UI/UX improvements, and feedback-driven iterations to ensure system usability and adoption.

5.1.2 Project Resources

1. **ML Engineer :** Led the design of the overall system architecture, including image preprocessing, text extraction, and ML model integration. Handled backend development to connect the mobile app with classification services and ensure smooth data flow. Took charge of dataset preparation, model training using XGBoost and Random Forest, and analyzing results. Also contributed to writing technical documentation and managing integration between frontend and backend components.
2. **App Developers:** Led the collection of food label images from various packaged food products to create a custom dataset from scratch, ensuring a mixed and representative sample of ultra-processed foods. Also, contributed in the development of the Flutter-based mobile application, implementing important features such as image capture, user-friendly input flow, and visual display of Nutri-Score results.

3. **Project Management Tool:** *Jira* will be utilized for tracking project tasks, managing workflows, and facilitating communication among team members. It will help in organizing sprints, monitoring progress, and ensuring that the project stays on schedule.
4. **Collaboration Tools:** *GitHub* for version control and collaborative coding, *Google Meet* for team meetings and discussions.

5.2 Risk Management

5.2.1 Risk Identification

Risk management in ultra-processed food classification using computer vision and machine learning involves identifying and addressing potential challenges to project success. Key risks include:

- **Data Quality Issues:** OCR errors, blurry images, and mislabelled nutritional tables may affect model inputs and prediction accuracy.
- **Model Generalization Failure:** Models like Random Forest and XGBoost might overfit specific packaging patterns or brand styles, leading to poor performance on unseen products.
- **Integration Complexity:** Integrating multiple technologies (YOLO, OCR, Regex, CTPN, TFLite, Flutter) could lead to compatibility issues or communication failures between modules.
- **Real-time Processing Limitations:** Processing time for capturing images, extracting text, and predicting Nutri-Score may be slow on lower-end mobile devices.
- **User Experience Risk:** Poor usability or UI glitches in the Flutter app may lead to user drop-off or incorrect usage.
- **Compliance and Mislabeling Risks:** Misclassifying food categories could have ethical or legal implications if users make dietary decisions based on inaccurate predictions.

Proactive risk management strategies, including regular evaluations, fallback workflows, and clear stakeholder communication, are essential for addressing these risks effectively.

5.2.2 Risk Analysis

Once risks are identified, a risk analysis will be conducted to assess their likelihood and potential impact. This includes:

- Evaluating risks based on probability, severity, and potential consequences for the system and users.
- Ranking risks by their influence on system accuracy, user trust, and project timelines.

- Developing contingency plans for critical risks like OCR failures or model misclassifications.
- Conducting regular risk assessments during key stages: image preprocessing, model training, integration, and mobile testing.

5.2.3 Overview of Risk Mitigation, Monitoring, and Management

- **Technical Risk Mitigation:** Continuous integration and testing of the complete pipeline – including YOLO, OCR, and CTPN – will be done with synthetic and real-world datasets to ensure module compatibility and robustness.
- **Data Quality Risk Mitigation:** Use of advanced preprocessing techniques (like image sharpening and binarization) and OCR correction using SymSpell and regex post-processing ensures cleaner data for model input.
- **Overfitting Risk Mitigation:** Random Forest and XGBoost models will be validated using cross-validation and a separate test set with diverse product images. Hyperparameter tuning and dropout-based ensembling techniques will be applied.
- **Integration and Resource Risk Mitigation:** All components will be tested within a simulated app environment and deployed using TensorFlow Lite (TFLite) for mobile compatibility, ensuring low memory consumption and smooth performance.
- **User Experience Risk Mitigation:** Multiple UI/UX testing phases with feedback loops from potential users will guide design changes. App behavior in case of OCR failure or missing fields will be handled with user-friendly prompts.
- **Compliance and Ethical Risk Mitigation:** Clear disclaimers about the limitations of the app will be added. Misclassified cases will be logged and retrained. No personal data will be collected, ensuring ethical deployment standards.

5.3 Project Schedule

5.3.1 Project Task Set

- **Requirement Analysis:** This phase involves gathering system and user requirements for the application, including functional components like image capture, object detection, OCR, text correction, and food classification. It also includes identifying necessary tools such as YOLO, Tesseract OCR, SymSpell, Flutter, TensorFlow Lite, and other third-party libraries for implementation.
- **Data Collection and Preprocessing:** Images of packaged food labels are collected and organized for training. The data is then pre-processed through resizing, noise removal, and contrast enhancement. Augmentation and manual correction ensure dataset diversity and improved performance in real-world scenarios.
- **Object Detection and OCR:** YOLO is used to detect the nutrition table region from the image. Once detected, OCR is applied to extract textual information from that region. Post-processing with regex and SymSpell corrects common OCR errors, ensuring clean and structured output.
- **Text Line Detection and Post-Processing:** CTPN is used to extract multi-line text from nutrition tables. This enhances line-wise segmentation and helps organize the extracted content into structured nutritional fields like energy, fat, sugar, fiber, and more, ready for classification.
- **Classification Model Development:** Machine learning models such as Random Forest and XGBoost are developed and trained using structured nutritional data. These models predict the Nutri-Score (A to E), and the most accurate one is optimized for mobile deployment.
- **Mobile App Development:** A cross-platform mobile application is developed using Flutter. The app allows users to capture food label images, view predictions, and interact with a clean and responsive user interface for a seamless user experience.
- **Integration and Conversion:** All components—YOLO, OCR, text processing, and classification—are integrated into a backend pipeline. The final model is converted to TensorFlow Lite for lightweight deployment and is connected to the Flutter frontend for real-time inference.

- **Testing and Validation:** Each module undergoes unit and system testing. The application is validated on a variety of real-world food labels. User testing is also conducted to gather feedback and ensure the robustness and reliability of the final system.

5.3.2 Task Network

Table 5.1: Project Timeline

The following table outlines the timeline of key tasks from project inception to completion.

SR NO.	TASK	LABOUR (HOURS/DAYS)
1	Project Planning and Survey for UPF Consumption	2 weeks
2	Database Creation & Trend Analysis from Survey Responses	2 weeks
3	Data Analysis and Feature Engineering	2 weeks
4	Designing Parameters and Selecting Algorithms for Prediction	1 week
5	Model Development and Training (Random Forest, XGBoost)	1 month
6	Implementing OCR, Regex, SymSpell, and CTPN for Text Extraction	3 weeks
7	Frontend Development using Flutter and Integration with TFLite	1 month
8	Testing on Unseen Data and Accuracy Improvement	2 weeks
9	Final Testing, Deployment, and Real-world Evaluation	2 weeks
10	Documentation, Analytics Extension, and Final Presentation	1 week

5.4 Team Organization

5.4.1 Team Structure

- **System Architecture and Machine Learning Team:** Responsible for designing the overall system including image preprocessing, text extraction, and integration of machine learning models for food classification. This team also manages backend development to ensure smooth communication between the mobile application and classification services. Additionally, they handle dataset preparation, model training

using algorithms like XGBoost and Random Forest, and the analysis of results. They contribute to technical documentation and ensure seamless integration between frontend and backend components.

- **Data Collection and App Development Team:** Focused on gathering food label images from various packaged food products to create a diverse and representative dataset of ultra-processed foods. This team structures the collected data to aid in efficient model training. They also lead the development of the Flutter-based mobile application, implementing features like image capture, secure user authentication, and intuitive display of Nutri-Score classifications. The team plays a key role in user experience design and integration of mobile interface components with machine learning outputs.
- **Frontend and User Interface Development Team:** Responsible for developing the mobile application using Flutter with an emphasis on user-friendly interfaces and secure user authentication via Firebase. This team ensures that input flows, result displays, and overall app navigation are smooth and intuitive. They also contribute to preprocessing data using regex and NLP techniques, helping enhance label extraction accuracy. The team documents the user experience and system workflow in the Methodology and System Architecture sections.

5.4.2 Management Reporting and Communication

- Project progress will be monitored through regular team meetings and status updates.
- Communication between teams will be facilitated using project management and collaboration tools such as Jira, GitHub, Google Meet.
- Reporting will include weekly summaries, milestone reviews, and documentation of key decisions and action items.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Data Collection Module:** This module involves conducting a survey to collect information about ultra-processed food (UPF) consumption patterns. It also includes gathering food label images from various packaged products to create a comprehensive and diverse dataset. The collected data will serve as the foundation for model training and classification.
- **Data Analysis and Preprocessing Module:** In this module, the gathered survey and label data are analyzed to identify trends and eating habits. Preprocessing steps such as text extraction using OCR, cleaning, and applying NLP techniques are performed to prepare the data for training machine learning models.
- **Classification Model Development Module:** Here, suitable machine learning algorithms like XGBoost and Random Forest are selected and trained on the processed dataset. This module focuses on developing a robust food classification model that assigns Nutri-Scores to products based on extracted information.
- **Model Testing and Optimization Module:** This module involves evaluating the trained models for accuracy, generalization to unseen data, and fine-tuning parameters to improve performance. Regular validation and testing cycles are carried out to ensure high classification reliability.
- **Mobile Application Development Module:** A Flutter-based mobile application is developed to allow users to capture food label images and view Nutri-Score results. The app integrates computer vision (CV) and NLP components, provides a secure user authentication system via Firebase, and displays results in a user-friendly way.
- **Integration and Deployment Module:** This module integrates the backend classification services with the mobile application. It ensures seamless communication between the app and the prediction models. The final product is deployed for real-world usage, allowing live testing and feedback collection.
- **7. Big Data Expansion and Analytics Module:** An optional extension module that focuses on scaling the database for big data analytics, enabling broader studies on food consumption trends and public health insights based on user-submitted data.

6.2 Tools and Technologies Used

- **Programming Languages:** Python, Dart (Flutter)
- **Frameworks and Libraries:** XGBoost, Scikit-learn, OpenCV, TensorFlow Lite, Flutter SDK
- **Models:** XGBoost Classifier, Random Forest Classifier, Optical Character Recognition (OCR) using Tesseract
- **Web Technologies:** Firebase Authentication
- **Deployment Environment:** Firebase Realtime Database for data storage, Android mobile devices for app deployment, local servers for initial model testing

6.3 Algorithm Details

6.3.1 Algorithm 1: YOLO

YOLOv5 was used in the project for object detection to locate the nutritional table and food label from images. It offered a good trade-off between accuracy and speed, enabling real-time detection on mobile devices. The model's use of CSPDarknet and data augmentation techniques made it well-suited for varied lighting and image quality in food packet images.

6.3.2 Algorithm 2: Tesseract OCR

Tesseract OCR was used for character recognition of the text detected by YOLO. It helped in extracting numeric values and labels from the nutritional table. The tool performed well with post-processing using regex and SymSpell, which corrected OCR errors and cleaned the extracted data for further analysis.

6.3.3 Algorithm 3: CTPN (Connectionist Text Proposal Network)

CTPN was employed to enhance text region detection, especially in noisy backgrounds or overlapping text scenarios. It helped in verifying the accuracy of bounding boxes around text regions, acting as an additional verification layer for OCR outputs and increasing reliability of text extraction.

6.3.4 Algorithm 4: Random Forest

The Random Forest classifier was trained on the extracted and cleaned nutritional data to predict the Nutri-Score (A to E). Its ensemble nature provided robustness against overfitting and was particularly effective for handling mixed types of nutritional features such as fat, sugar, and fiber content.

6.3.5 Algorithm 5: XGBoost

XGBoost was used alongside Random Forest for Nutri-Score classification due to its ability to model complex non-linear relationships and handle missing values. It gave high accuracy and interpretability for feature importance, which helped in validating the model's focus on health-critical ingredients.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

7.1.1 Frontend Testing

- **UI/UX Functionality Testing:** Verified that all UI components such as buttons, image capture interface, and result display work smoothly across devices. Ensured intuitive navigation, proper alignment, and responsive design for varied screen sizes.
- **Input Validation and Feedback:** Tested user inputs like image uploads and manual entries to check error handling (e.g., unreadable images or empty fields). Ensured appropriate error messages and success feedback are provided.
- **Integration with Backend Services:** Confirmed that the frontend correctly communicates with the ML classification model and Firebase. Validated data flow from image input to result display, including authentication, prediction, and result rendering.

7.1.2 Backend Testing

- **Model Integration and Pipeline Execution:** Verified that all components of the back-end pipeline from image preprocessing, nutrition table detection (YOLOv11), OCR extraction, text cleaning, feature extraction and produce consistent intermediate results.
- **Prediction Accuracy and Consistency:** Evaluated the response from both Random Forest and XGBoost classifiers. Ensured the backend correctly loads the trained models, processes input features, and returns the appropriate class label (A, B, C, D) with high consistency.
- **Error Handling and Robustness:** Conducted tests on corrupted images, missing labels, and partial nutrition values. Confirmed that the back-end gracefully handles exceptions, returns informative error messages, and avoids crashes during runtime.
- **File and Model Management:** Checked that the YOLO model (‘.pt’ file) and XGBoost pickle file (‘.pkl’) load correctly at runtime. Verified predictions are logged and saved in CSV format (‘predictions.csv’) without data overwrites.

ML based classification of Ultra Processed Foods using Computer Vision,
NLP and Big Data

- API Communication: If used with a Flask API, ensured that HTTP POST requests were correctly handled and that responses to the frontend were properly structured in JSON format.

7.2 Test cases & Test Results

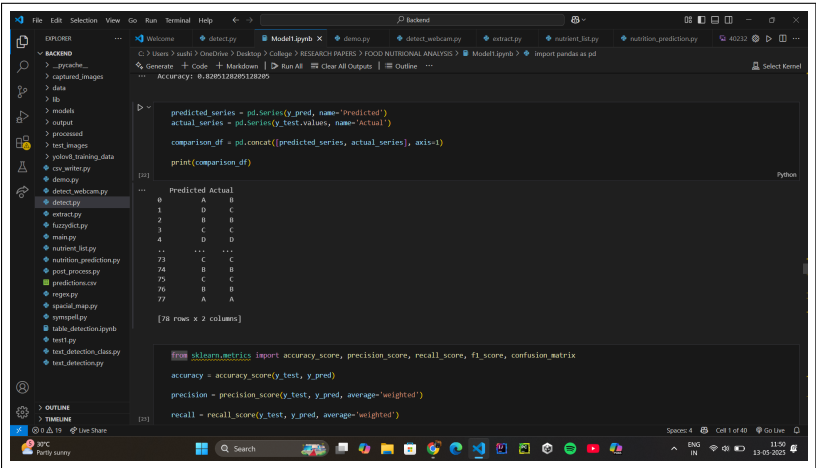


Figure 7.1: Predictions and Actual values

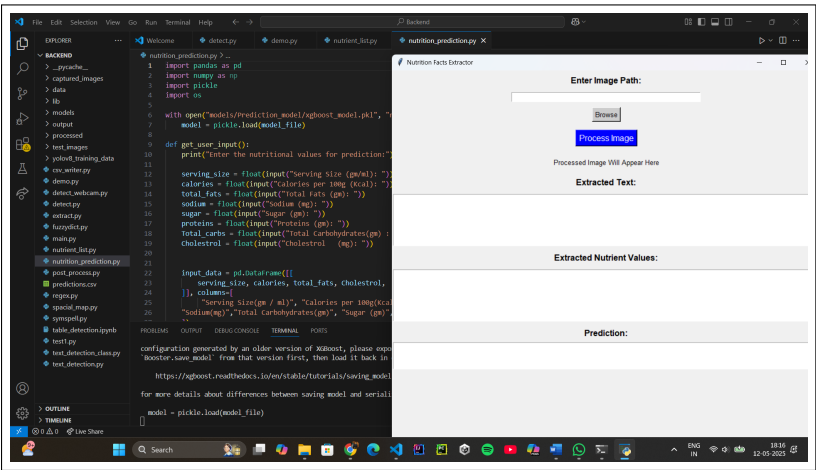


Figure 7.2: File and Model Management

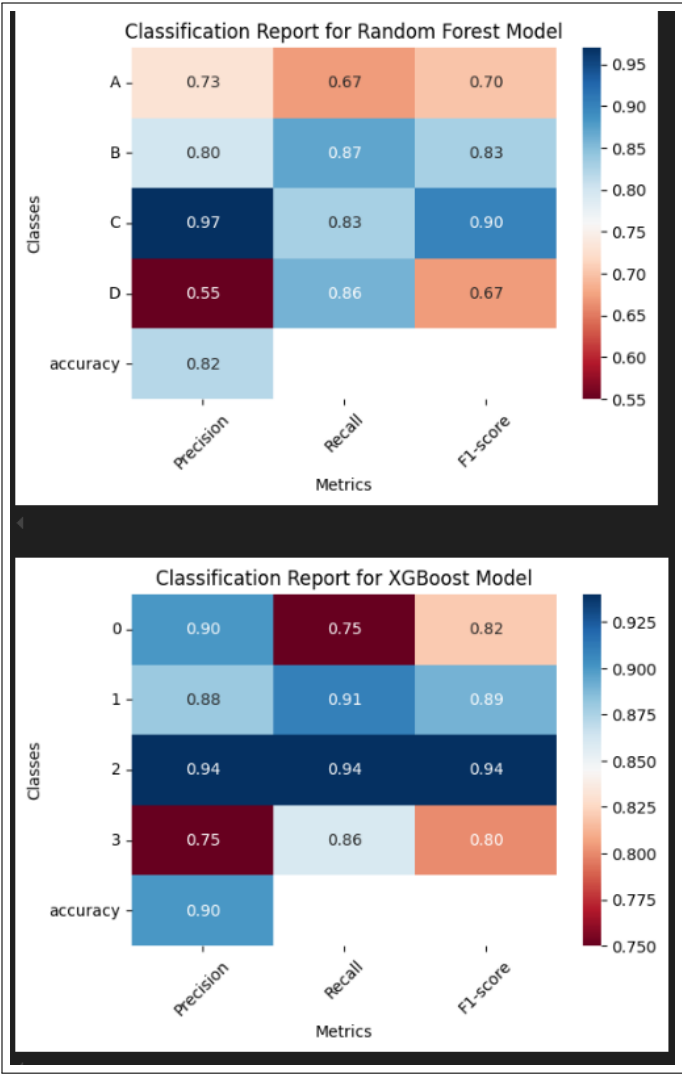


Figure 7.3: Classification Report of Models

ML based classification of Ultra Processed Foods using Computer Vision,
NLP and Big Data

Serve Size- 50g Pack contains 6 (approx.) serves

Nutritional Information	Per 100g	Per Serve	%RDA Per Serve
Energy (kcal)	210.2	105.1	5.3%
Protein (g)	2.1	1.1	1.9%
Carbohydrate (g)	24.7	12.4	--
-Total Sugars (g)	19.4	9.7	--
-Added Sugars (g)	14.5	7.3	14.5%
Total Fat (g)	11.4	5.7	8.5%
-Saturated Fat (g)	1.9	1.0	4.3%
-Trans Fat (g)	0.1	0.1	2.5%
-Cholesterol (mg)	0.0	0.0	--
Sodium (mg)	1650.2	825.1	41.3%

*Per Serve Recommended Dietary Allowances (RDA) basis 2000 kcal energy

Figure 7.4: Test Case 1

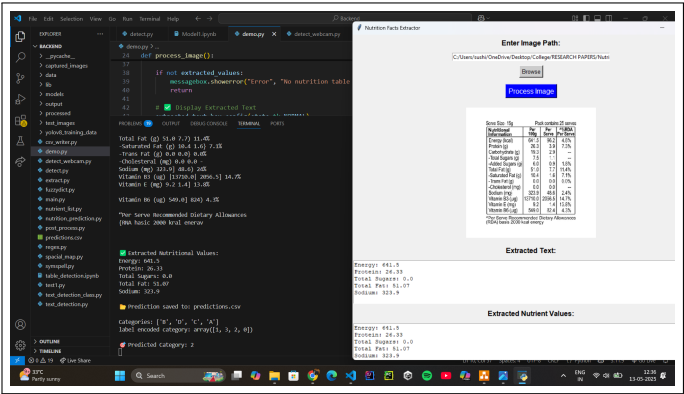


Figure 7.5: Test Case 1 Result

CHAPTER 8

RESULTS

8.1 Outcomes

1. The system was able to accurately detect nutrition tables using YOLOv8 and extract relevant text using OCR with preprocessing.
2. Most noisy OCR outputs were corrected using regex and fuzzy matching to ensure valid feature extraction.
3. Classification models (XGBoost and Random Forest) predicted food categories with consistent accuracy across multiple test cases.

8.2 Screen Shots

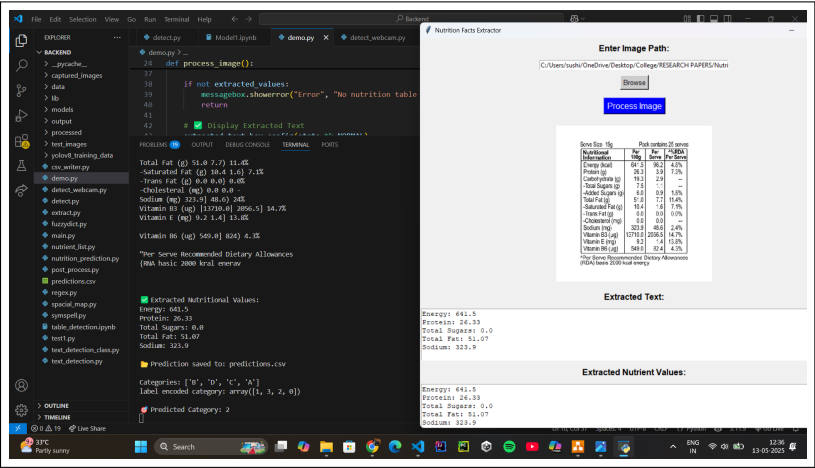


Figure 8.1: Predictions results on seen label

•

```
Information 100g | Serve | Per Serve
Energy (kcal) 641.5) 96.2) 48%

Protein (g) 26.3 3.9] 7.3%
Carbohydrate (g) 19.3 29 -
-Total Sugars (g) 75 11 -
-Added Sugars {g} 6.0 0.9) 1.8%

Total Fat (g) 51.0 7.7) 11.4%
-Saturated Fat (g) 10.4 1.6} 7.1%
-Trans Fat (g) 0.0 0.0} 0.0%
-Cholesterol (mg) 0.0 0.0 -
Sodium (mg) 323.9] 48.6) 24%
Vitamin B3 (ug) [13710.0| 2056.5] 14.7%
Vitamin E (mg) 9.2 1.4] 13.8%

Vitamin B6 (ug) 549.0] 824) 4.3%

"Per Serve Recommended Dietary Allowances
{RNA hasic 2000 kcal enerav

✅ Extracted Nutritional Values:
Energy: 641.5
Protein: 26.33
Total Sugars: 0.0
Total Fat: 51.07
Sodium: 323.9

📁 Prediction saved to: predictions.csv

Categories: ['B', 'D', 'C', 'A']
label encoded category: array([1, 3, 2, 0])

🎯 Predicted Category: 2
□
```

Figure 8.2: Extracted text on seen label

ML based classification of Ultra Processed Foods using Computer Vision, NLP and Big Data

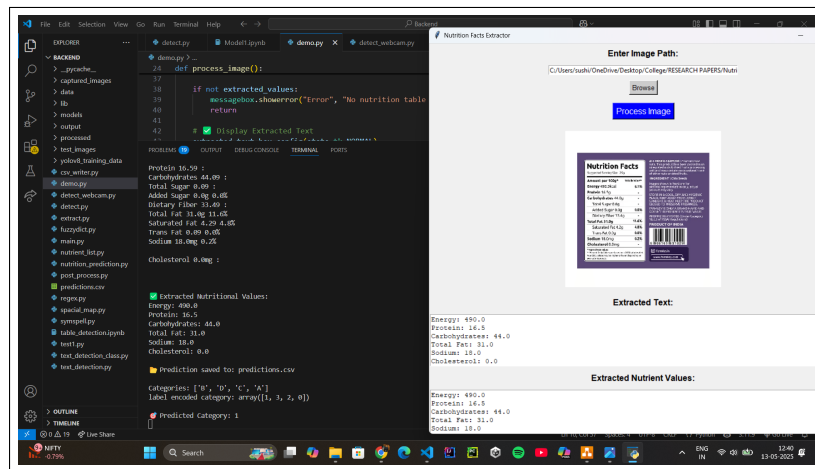


Figure 8.3: Predictions on random food label

```
PROBLEMS 19 OUTPUT DEBUG CONSOLE TERMINAL PORTS

📄 Extracted Raw Text:
Nutrition Facts

Suggested Serving Size 25g

Amount per 100g* — %ailyvaluew

Energy 490.0kcat 61%
Protein 16.59 :
Carbohydrates 44.09 :
Total Sugar 0.09 :
Added Sugar 0.0g 0.0%
Dietary Fiber 33.49 :
Total Fat 31.0g 11.6%
Saturated Fat 4.29 4.8%
Trans Fat 0.09 0.0%
Sodium 18.0mg 0.2%

Cholesterol 0.0mg :

✅ Extracted Nutritional Values:
Energy: 490.0
Protein: 16.5
Carbohydrates: 44.0
Total Fat: 31.0
Sodium: 18.0
Cholesterol: 0.0

📁 Prediction saved to: predictions.csv

Categories: ['B', 'D', 'C', 'A']
label encoded category: array([1, 3, 2, 0])

🔗 Predicted Category: 1
```

Figure 8.4: Extracted text on random food label

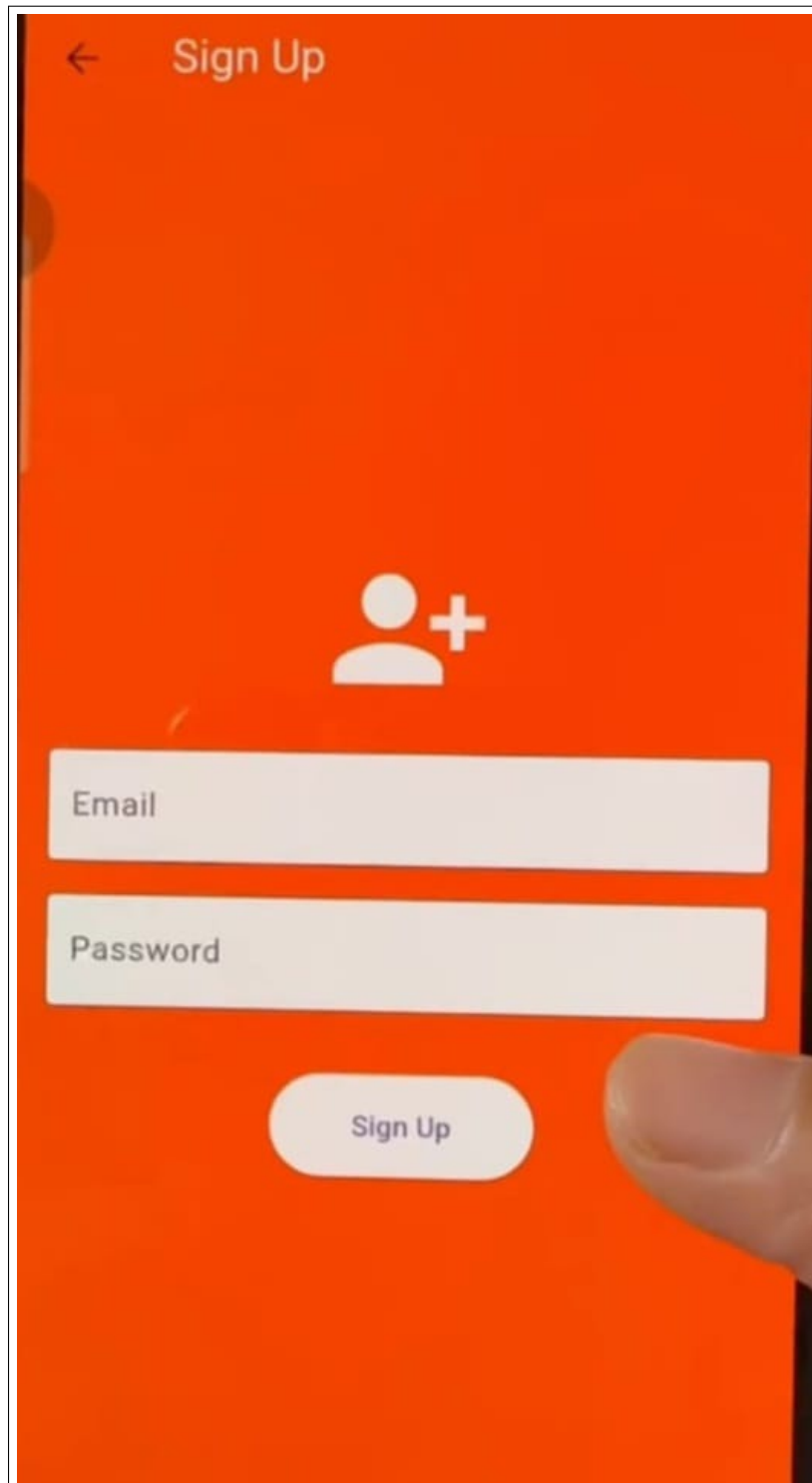


Figure 8.5: Sign Up Page

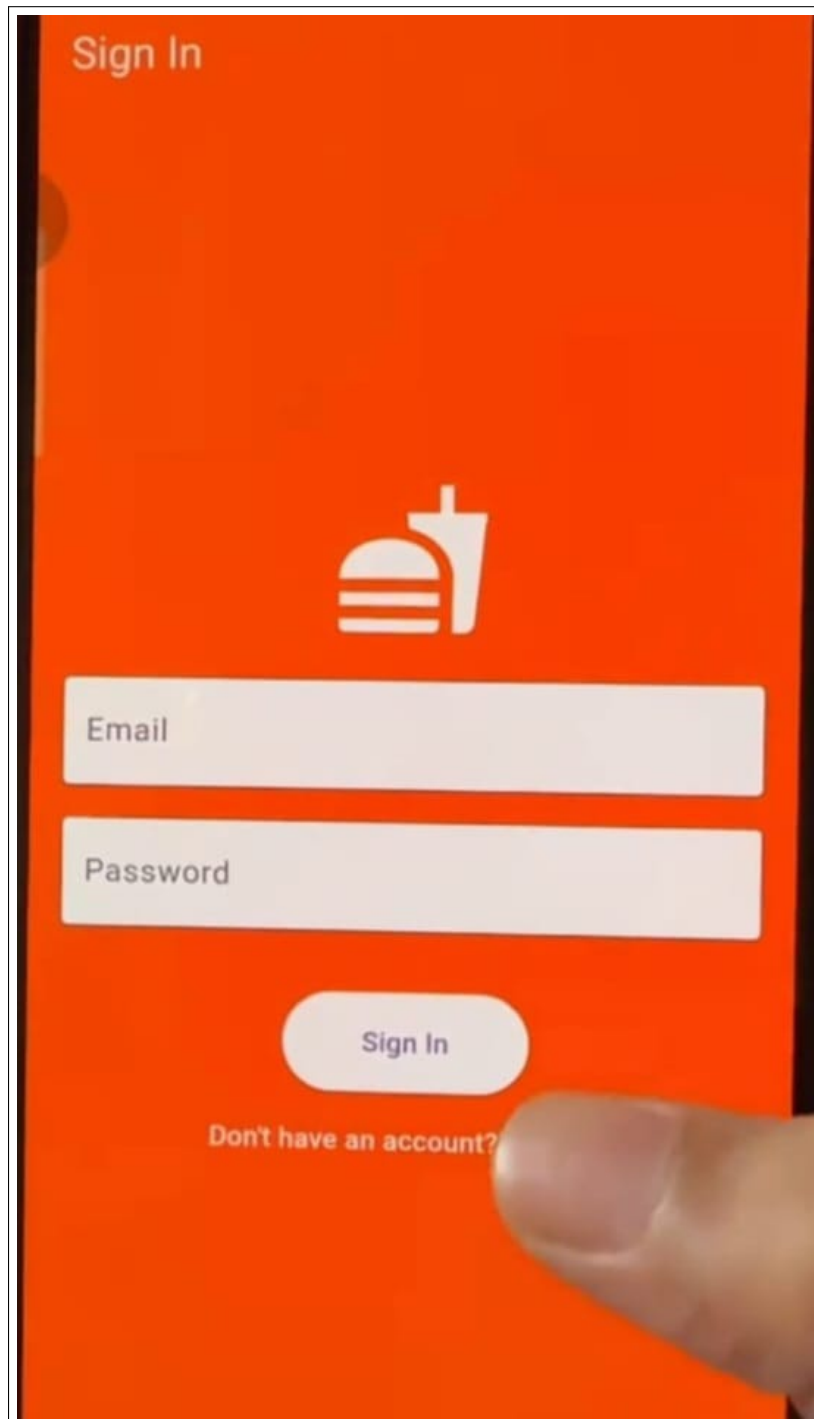


Figure 8.6: Sign In Page

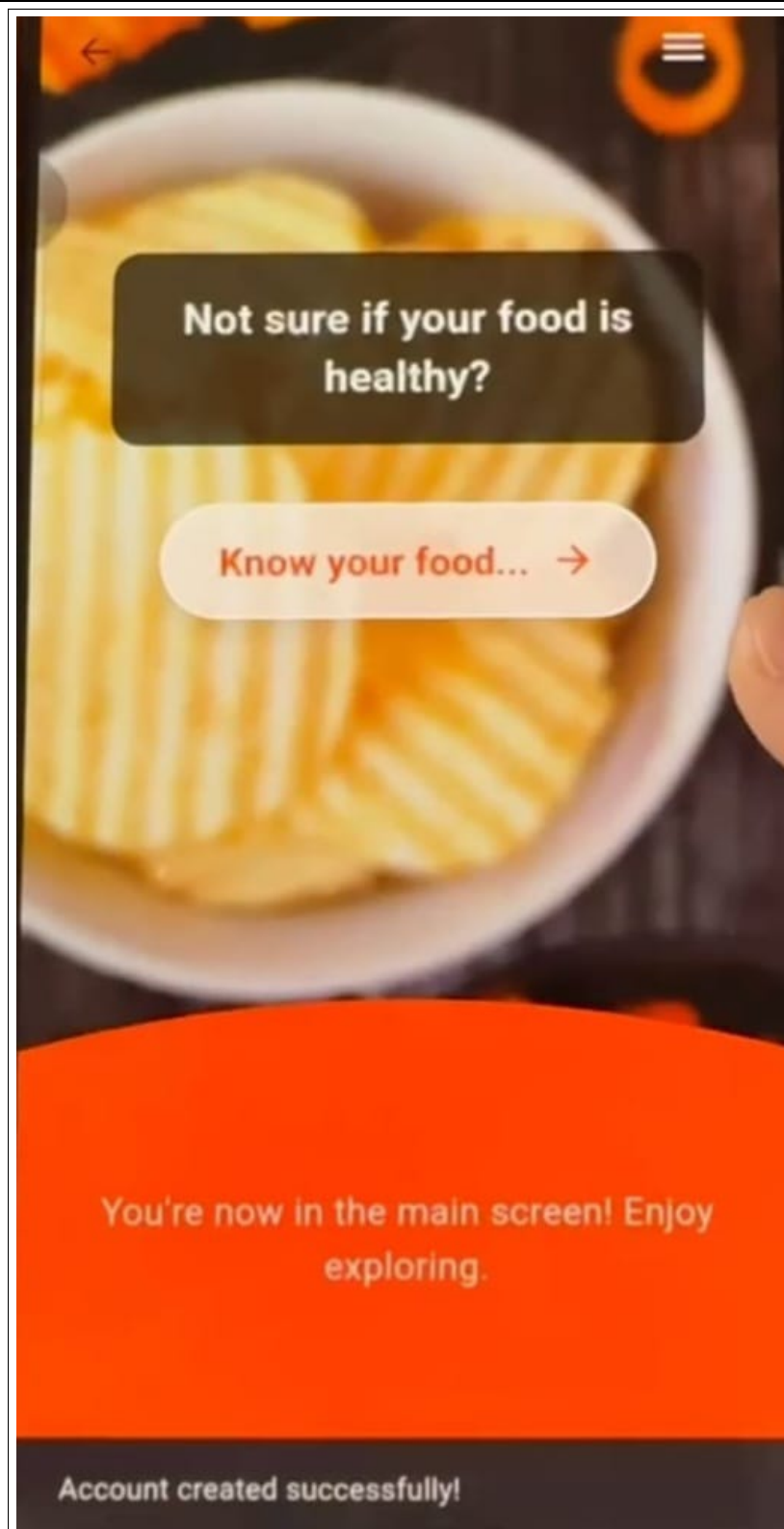


Figure 8.7: Landing Page



Figure 8.8: Image Capturing Page

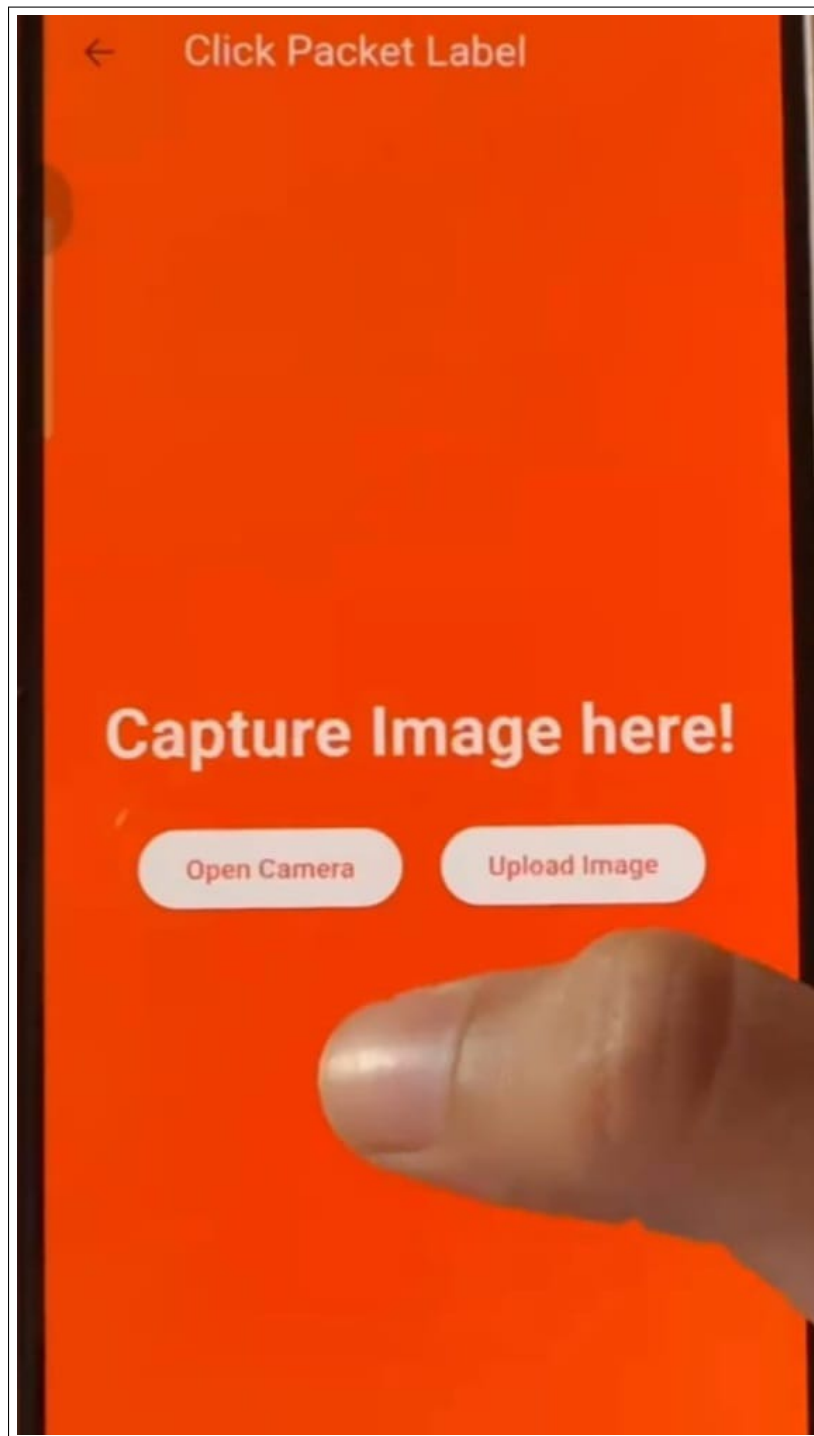


Figure 8.9: Clicking the image

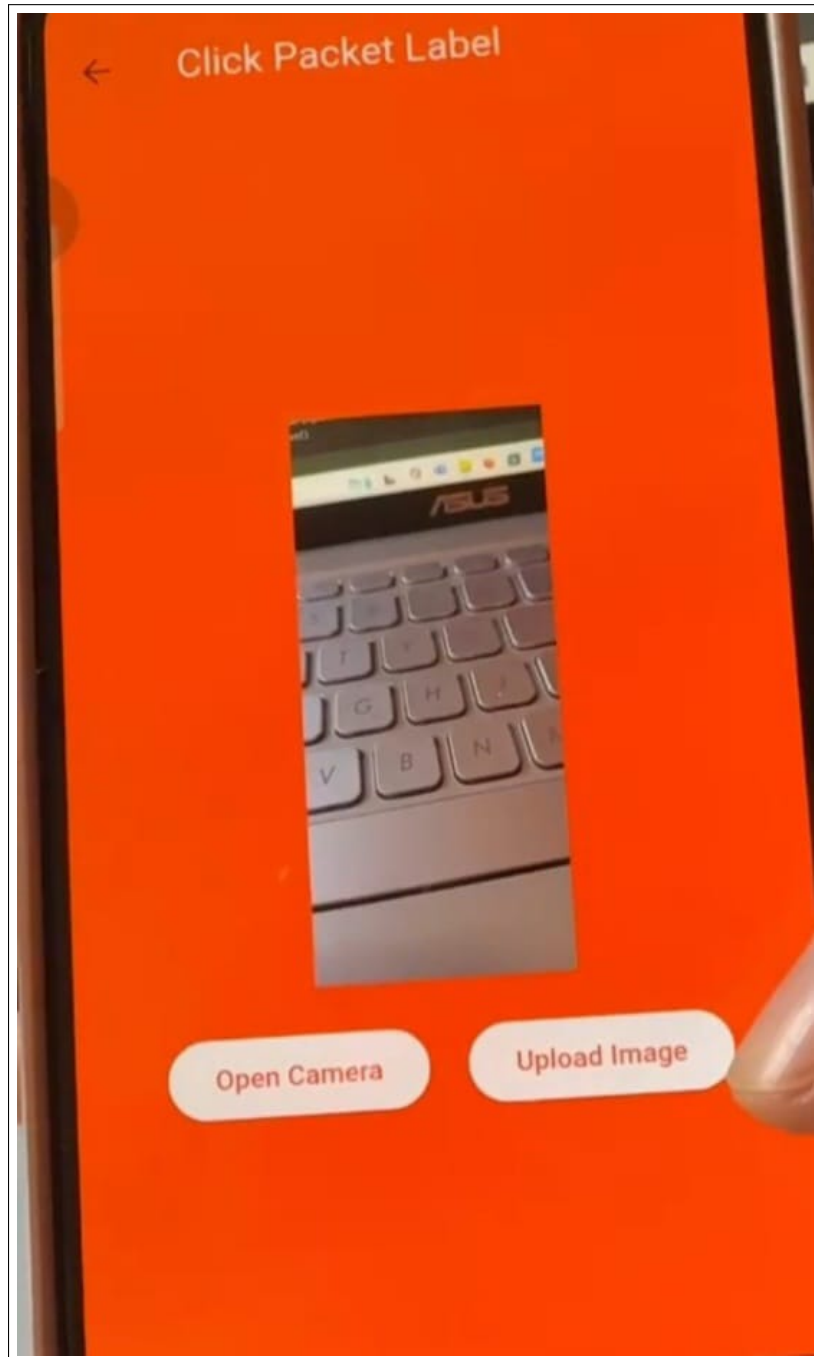


Figure 8.10: Image Upload

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

This project successfully demonstrates the application of machine learning for public health by automating the classification of processed food products using the Nutri-Score system. Through the integration of mobile image capture, OCR-based text extraction, and ensemble models like XGBoost and Random Forest, users receive real-time feedback on food quality. The Flutter-based mobile app ensures an accessible and user-friendly experience. Our pipeline—from image preprocessing to model inference—was designed for scalability and adaptability. Initial testing confirms the system’s accuracy and reliability in controlled conditions. This solution empowers users to make informed dietary choices and can serve as a foundation for broader food health awareness platforms.

9.2 Conclusions

The project effectively addresses the growing concern over processed food consumption by providing users with immediate and informative Nutri-Score classifications. By combining computer vision, natural language processing, and machine learning, the system simplifies complex nutritional data into accessible health indicators. The user-friendly mobile interface allows for quick scanning of food labels and delivers clear feedback, making the app both practical and impactful. This contributes significantly to increasing consumer awareness and promoting healthier eating habits. Overall, the project demonstrates a meaningful application of technology to support individual well-being and public health initiatives.

9.3 Future Work

1. **Multilingual Label Support:** Given the diversity of languages on food packaging, especially in regions like India, future versions of the system should support nutrition labels in multiple regional and international languages. Incorporating OCR capabilities for languages such as Hindi, Marathi, Tamil, or Spanish can significantly expand the reach and usability of the application globally.

2. **Personalized Dietary Recommendations:** Beyond classification, the system can integrate a recommendation engine that suggests healthier alternatives or personalized dietary tips based on the user's scan history and preferences. This can empower users not only to avoid unhealthy food but to make proactive, informed nutritional decisions.
3. **Active Learning and Feedback Loop:** To continuously improve model accuracy and adaptability, the app can implement an active learning pipeline where user feedback on classification results is incorporated into retraining cycles. This user-in-the-loop approach helps in reducing prediction errors over time and aligns the model closer to real-world use cases.
4. **Integration with Government and Health Programs:** The system can be scaled and aligned with public health initiatives or integrated with food safety authorities and nutritional awareness campaigns. With appropriate partnerships, it could serve as a backend for large-scale food monitoring or dietary tracking systems.
5. **Cloud-based Analytics Dashboard:** A companion web dashboard for researchers, health professionals, or administrators can be developed to visualize data trends, user habits, and food classification statistics in real-time. This would transform the app into a powerful tool for big data analytics in food consumption behavior.
6. **Cross-platform and Smartwatch Integration:** Expanding beyond mobile, future iterations can be made compatible with wearables or smart home devices (e.g., smart fridges or voice assistants), enabling seamless nutritional tracking embedded into daily life routines.

9.4 Applications

1. **Consumer Health Awareness:** Helps consumers make informed dietary choices by instantly identifying the nutritional quality of packaged food products through Nutri-Score classification. Empowers users to avoid ultra-processed or unhealthy food items based on scientific scoring rather than brand claims.
2. **Educational Tool for Nutrition:** Can be used in schools or public health campaigns to educate people, especially children and adolescents, about reading food labels and understanding the impact of processed foods.

3. **Retail and Supermarket Integration:** Can be embedded into supermarket applications to offer on-the-spot nutritional ratings and healthier alternative suggestions while shopping. Enhances the smart shopping experience by integrating health-conscious decision-making.
4. **Public Health and Government Policy Implementation:** Supports large-scale monitoring and auditing of food product health standards by government bodies and food safety regulators. Can aid in enforcing regulations on labeling and promoting healthier food manufacturing practices.
5. **Dietician and Clinical Use:** Dietitians can use the app to assist clients in tracking and evaluating their packaged food consumption patterns. Useful in clinical nutrition settings to monitor patient diets in real-time.
6. **Research and Nutritional Studies:** Facilitates data collection for nutritional research by analyzing trends in processed food consumption and the effectiveness of Nutri-Score labeling

CHAPTER 10

APPENDIX

10.1 Appendix A

Problem Overview:

Modern machine learning frameworks such as *Scikit-learn*, *TensorFlow*, and *PyTorch* enable the implementation of intelligent food classification models using structured nutritional datasets.

The integration of supervised learning models and computer vision tools like OCR (e.g., *Tesseract* with *YOLOv3* or *EasyOCR*) demonstrates strong technical feasibility for extracting nutritional features and categorizing food items based on the Nutri-Score algorithm.

The project aims to classify processed and ultra-processed food products from nutritional labels into categories ranging from **A (most healthy)** to **E (least healthy)**, leveraging data such as calorie count, sugar, saturated fat, sodium, fiber, and protein content.

Operational Feasibility:

- The system was successfully tested using real-world datasets containing nutritional information of packaged food products sourced from open food repositories, supermarkets, and online grocery listings.
- A modular pipeline was implemented, integrating OCR for label extraction, preprocessing, feature calculation, and classification using Nutri-Score thresholds.
- A user-friendly interface was developed to upload images or datasets and view classified results along with color-coded Nutri-Score labels.
- Integration with platforms like Google Colab and GitHub allowed collaborative development, testing, and result visualization.

Economic Feasibility:

- The project makes use of open-source libraries and APIs, significantly reducing software development and deployment costs.
- Cost-benefit analysis indicates that consumers and nutrition researchers can benefit from quick and reliable assessments of food healthiness, enabling informed decision-making.
- Optional deployment on cloud platforms ensures scalability with minimal additional infrastructure costs.

- Economic value is expected through integration with health and wellness applications, dietary advisory tools, or as a plugin for e-commerce platforms to promote transparency in food choices.

Mathematical Model:

- **Nutri-Score Calculation:** A point-based scoring formula based on both negative factors (calories, sugar, saturated fat, sodium) and positive ones (fiber, protein, fruits/vegetables).
- **OCR Processing:** *YOLOv3* and *Tesseract/EasyOCR* models are used for object detection and text extraction from food labels.
- **Dimensionality Reduction:** PCA can be applied for analysis of high-dimensional nutrition datasets to improve classifier performance and eliminate noise.
- **Machine Learning Models:** Algorithms such as *Random Forest*, *SVM*, or *XGBoost* may be used to supplement or cross-validate the rule-based Nutri-Score outputs, especially when dealing with missing or noisy data.
- **Optimization:** Gradient descent, cross-validation, and hyperparameter tuning techniques are employed to improve classification accuracy.

Complexity Analysis:

- Preprocessing pipelines using *NumPy*, *Pandas*, and *OpenCV* ensure clean and structured data inputs.
- Parallel OCR processing and batch classification improve processing speed when dealing with large sets of food items.
- Model checkpointing and efficient memory usage were implemented for scalable deployment across systems.

NP-Hard or P-Type Problems:

- **NP-Hard Components:** Text extraction from inconsistent nutritional labels, estimating missing features from incomplete data, evaluating Nutri-Score over mixed units (per 100g/ml).

- **P-Type Components:** Nutri-Score assignment, label parsing, image-to-text conversion using YOLO + OCR.

The overall system balances complexity and computational feasibility through modular design and scalable deployment, while offering an accurate, interpretable, and user-friendly categorization tool for processed food products.

10.2 Appendix B

Survey Paper

- **Title:** ML Based Ultra Processed Food Categorization with Limited Data
- **Journal:** 8th International Conference on Computing, Communication, Control and Automation (ICCUBEA)
- **Status:** Published (ICCUBEA, December 2024)

Implementation Paper

- **Title:** Nutritional Quality Analyzer For Processed Foods Using ML
- **Journal:** Science Direct
- **Status:** Submission in process

10.3 Appendix C

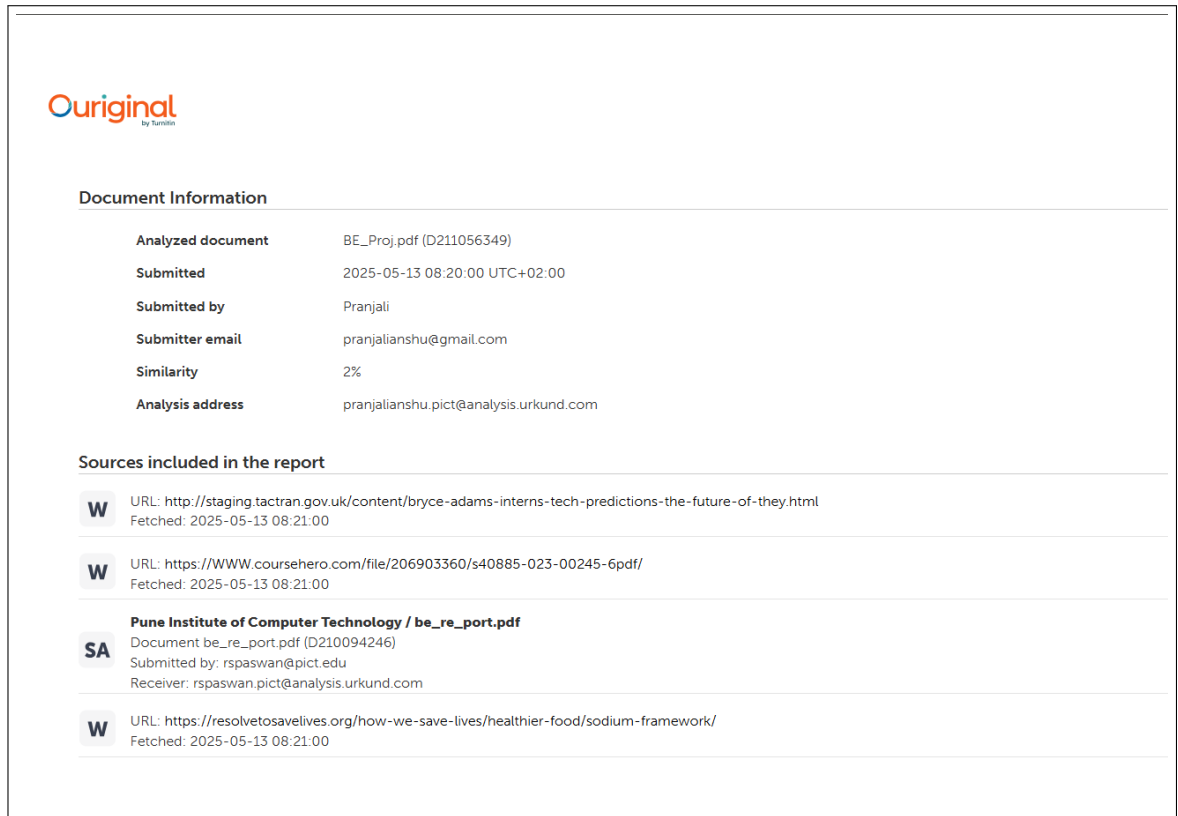


Figure 10.1: Plagiarism Report of BE Project Report

CHAPTER 11

REFERENCES

- [1] Aiello, L. M., Schifanella, R., Quercia, D., Del Prete, L. (2019). High resolution as well as large-scale analysis of health outcomes and food purchases. *EPJ Data Science*.
- [2] Singh, G. M., Micha, R., Khatibzadeh, S., Shi, P., Lim, S., Andrews, K. G., Engell, R. E., Ezzati, M., Mozaffarian, D. (2015, August 5). Global, regional, and national consumption of sugar-containing beverages, fruit juices, and milk. *Global Burden of Diseases Nutrition and Chronic Diseases Expert Group (NutriCoDE)*.
- [3] McCann, S. E., Marshall, J. R., Brasure, J. R., Graham, S., Freudenheim, J. L. (2001). Food categorization in principal components analytics and its effects on prediction for endometrial cancer: Analysis of patterns of food intake in nutritional epidemiology. Submitted January 4, 2001; Accepted March 6, 2001.
- [4] Anitha, E., Banu, A. B. (n.d.). A comprehensive review on analyzing the nutritional content of packed foods using smartphone camera/computer vision and machine learning.
- [5] Food Safety and Standards Authority of India. (2024). *Indian Food Reference Version II*. Retrieved from <https://www.fssai.gov.in/upload/uploadfiles/files/Comp>
- [6] Chen, T., Guestrin, C. (n.d.). Model of the working principle of the XGBoost algorithm. *ResearchGate*. Retrieved from https://www.researchgate.net/figure/Working-principle-model-of-the-XGBoost-algorithm_fig1_361193793
- [7] Wikipedia contributors. (2024, April 23). Nutri-Score. *Wikipedia*. Retrieved from <https://en.wikipedia.org/wiki/Nutri-Score>
- [8] Menichetti, G., Ravandi, B., Mozaffarian, D., Barabási, A.-L. (2023). Prediction of the degree of food processing using machine learning. *Nature Communications*, 14(1), 2312. <https://doi.org/10.1038/s41467-023-37457-1>
- [9] Silva, V. C., Gorgulho, B., Marchioni, D. M., de Araujo, T. A., Santos, I. d. S., Lotufo, P. A., Benseñor, I. M. (2022). Prediction of dietary patterns using clustering analysis and machine learning algorithms: Cross-sectional results of the Brazilian Longitudinal Study of Adult Health (ELSA-Brasil). *Journal of Human Nutrition and Dietetics*, 35(5), 883–894. <https://doi.org/10.1111/jhn.12992>

- [10] Katidi, A., Vlassopoulos, A., Noutsos, S., Kapsokefalou, M. (2023). Categorization of ultra-processed foods in the Mediterranean diet: A food-level analysis of branded foods in Greece, according to the NOVA classification system. *Foods*, 12(7), 1520. <https://doi.org/10.3390/foods12071520>
- [11] Karpe, S., Kulkarni, L., Patil, A., Joshi, P. (2024). ML-based ultra-processed food categorization with limited data. In *2024 8th International Conference on Computing, Communication, Control and Automation (ICCUBE)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICCUBE61740.2024.10774784>
- [12] Hu, G., Ahmed, M., L'Abbé, M. R. (2023). Comparison of natural language processing and machine learning approaches with traditional methods for food categorization and nutrition quality prediction. *The American Journal of Clinical Nutrition*, 117(3), 553–563. <https://doi.org/10.1016/j.ajcnut.2022.11.022>
- [13] Abreu, S., Martins, M. L. (2023). Two-way classification and analytics of food products based on degree of processing and nutritional quality. *Nutrients*, 15(14), 3117. <https://doi.org/10.3390/nu15143117>
- [14] Fondevila-Gascón, J.-F., Berbel-Giménez, G., Vidal-Portés, E., Hurtado Galarza, K. (2022). Implementing Nutri-Score to make healthy choices for ultra-processed foods in university students. *Healthcare*, 10(6), 984. <https://doi.org/10.3390/healthcare10060984>
- [15] Sailesh, R., Subiksha, S., Yamini, R., Vijayalakshmi, Naveenkumar, K. (2024). OCR using YOLOV3 and Tesseract for text extraction. *International Journal of Innovative Research in Technology (IJIRT)*, 11(5), ISSN: 2349-6002.
- [16] Kishor, R. (2024). Performance benchmarking of YOLOv11 variants for real-time delivery vehicle detection: A study on accuracy, speed, and computational trade-offs. *Asian Journal of Research in Computer Science*, 17(12), 108–122. <https://doi.org/10.9734/ajrcos/2024/v17i12532>

LIST OF FIGURES

Figure	Illustration	Page No.
4.1	Model Architecture	18
4.2	Mathematical Model	19
4.3	Data Flow Diagram	21
4.4	ER Diagram	22
4.5	UML Diagram Level 0	23
4.6	Sequence Diagram Level 1	24
7.1	Predictions and Actual values	39
7.2	File and model management	39
7.3	Classification Report of models	40
7.4	Test Case 1	41
7.5	Test Case 1 Result	41
8.1	Predictions results on seen label	43
8.2	Extracted Text on seen label	44
8.3	Predictions on random food label	45

Figure	Illustration	Page No.
8.4	Extracted text on random food label	46
8.5	Sign Up Page	47
8.6	Sign In Page	48
8.7	Landing Page	49
8.8	Image Capturing Page	50
8.9	Clicking the Image	51
8.10	Image Upload	52
10.1	Plagiarism Report of BE Project Report	62

LIST OF TABLES

Table	Illustration	Page No.
5.1	Project Timeline	30